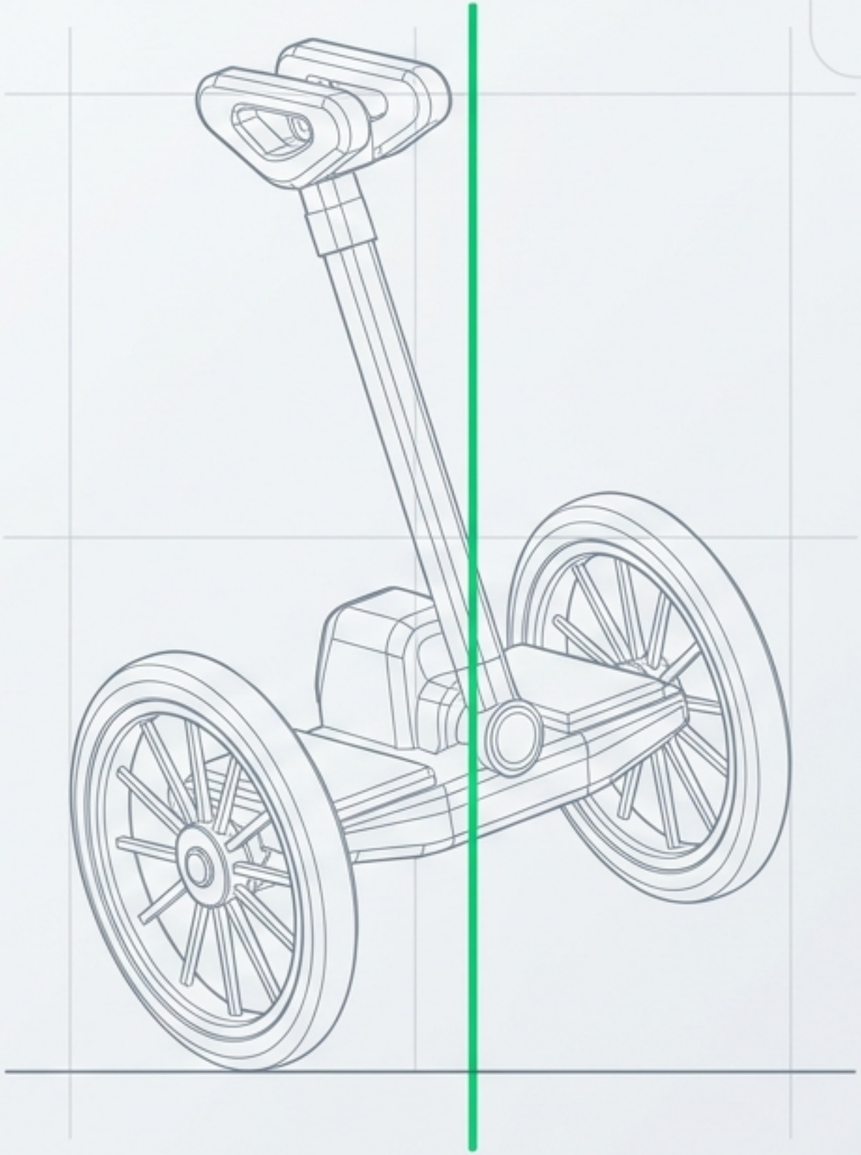
# Mastering the Balancing Act

## Mathematical Modeling, System Identification, and PID Control of a Two-Wheeled Robot



NotebookLM

# The Closed-Loop Physical Architecture

Node 1

## The Brain (Controller)

**Arduino UNO R3**

Executes the digital PID control algorithm based on real-time sensor feedback.

## Signal Flow Matrix

Node 2

## The Muscle (Actuation)

**L298N Motor Driver + 12V DC Motors**

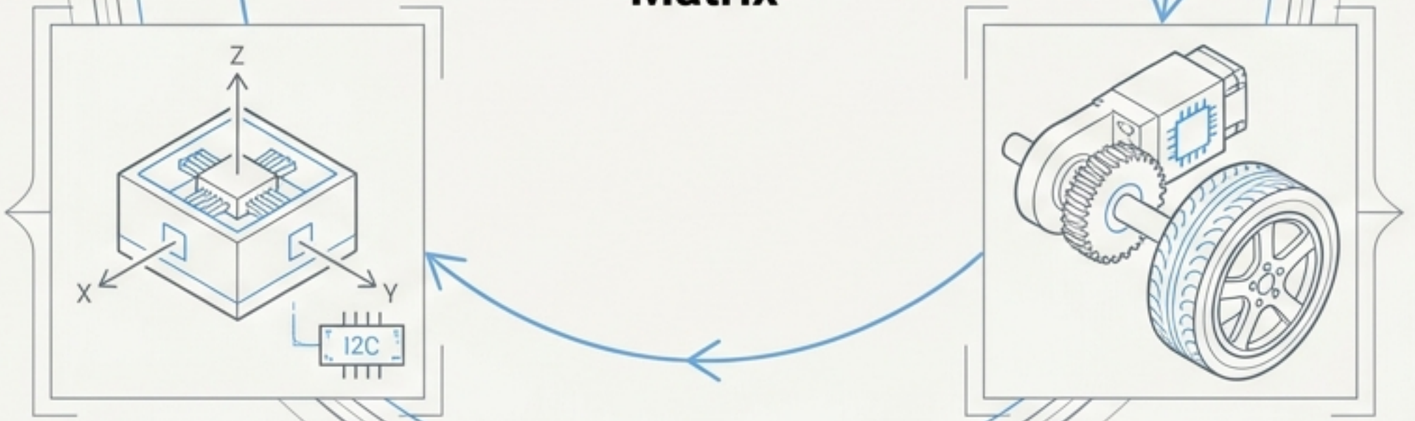
Translates the calculated PWM duty cycle (u) into restoring torque at the wheels.

Node 3

## The Senses (Feedback)

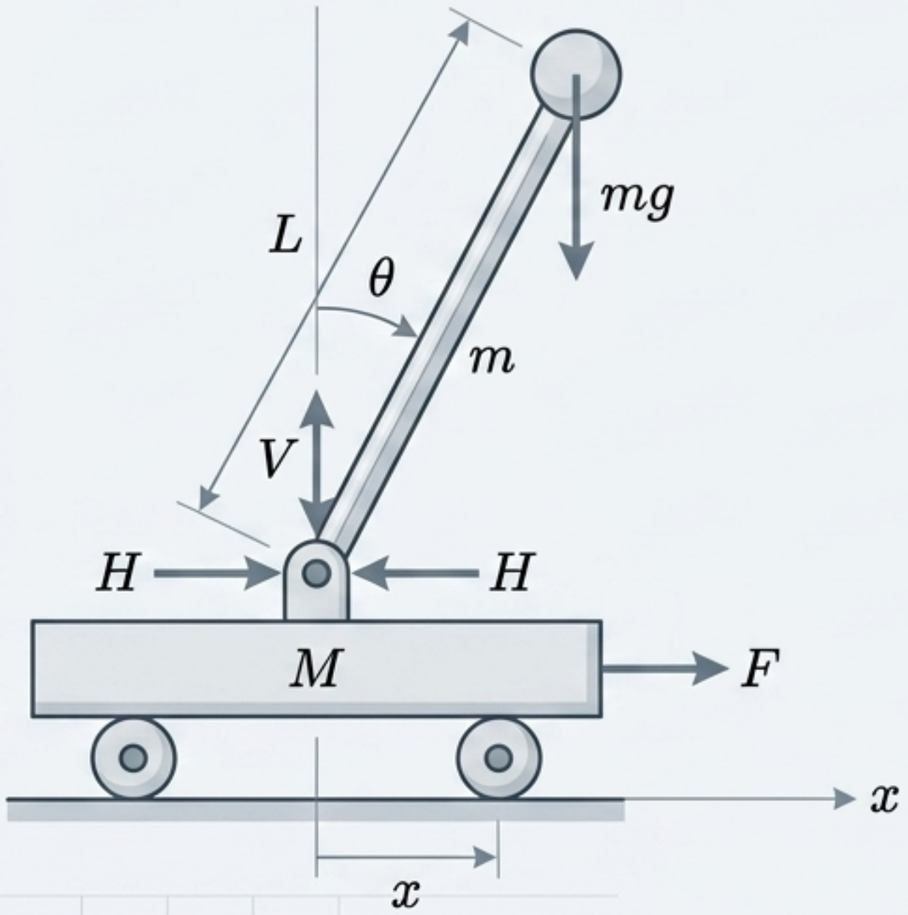
**MPU-6050 IMU**

Measures exact body tilt angle (θ) and angular rate ($\dot{\theta}$) via a 400 kHz I2C bus.




NotebookLM

# Formulating the Physics of Instability




Horizontal Cart Dynamics:

$$(M+m)\ddot{x} + b\dot{x} + mL\,\ddot{\theta}\cos\theta - mL\,\dot{\theta}^2\sin\theta = F$$

Tangential Pendulum Dynamics:

$$L\,\ddot{\theta} + g\sin\theta = \ddot{x}\cos\theta$$

### The Core Challenge

The system is **inherently unstable** and underactuated. Gravitational pull is constantly fighting against the horizontal reaction forces and viscous friction.

NotebookLM

# Linearizing the Mechanical Plant

**Input**

Non-linear Newtonian equations of motion.

**Process**

- Small-angle approximations ($\sin\theta \approx \theta, \cos\theta \approx 1, \dot{\theta}^2 \approx 0$).
- Integration of the 1st-order DC motor model ($\tau_e \ll \tau_m$).

**Output** (The 3rd-Order Transfer Function)

$$\frac{\Theta(s)}{U(s)} = \frac{K_m K_p}{\tau s^3 + (1+\beta\tau)s^2 + (\omega_0^2\tau + \beta)s - \omega_0^2}$$

**System Architecture:** One real pole from the motor dynamics, combined with two complex poles from the mechanical plant.

NotebookLM

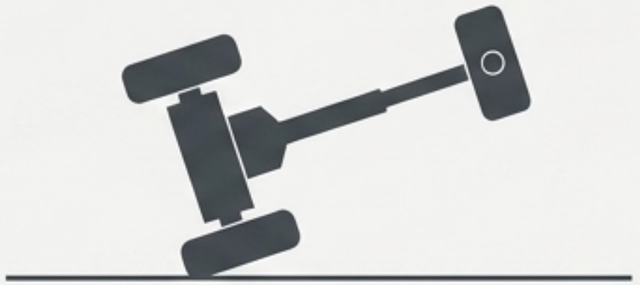
# System Identification

Bridging Theoretical Models
with Physical Hardware.

NotebookLM

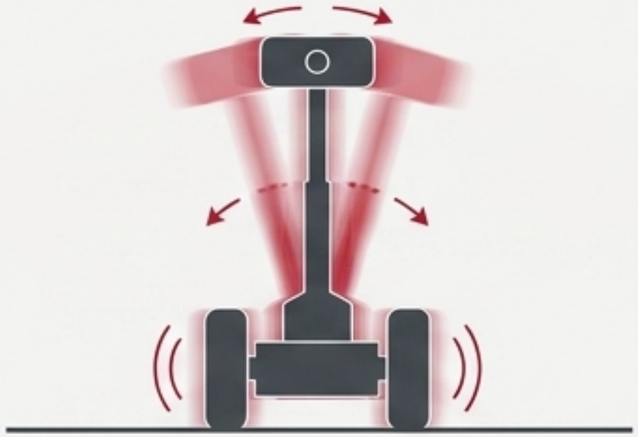
# The Data Collection Dilemma

## Zero Control = Dead Data



The Problem: An inherently unstable system cannot be characterized if it immediately collapses.

## Baseline PID = Dynamic Data



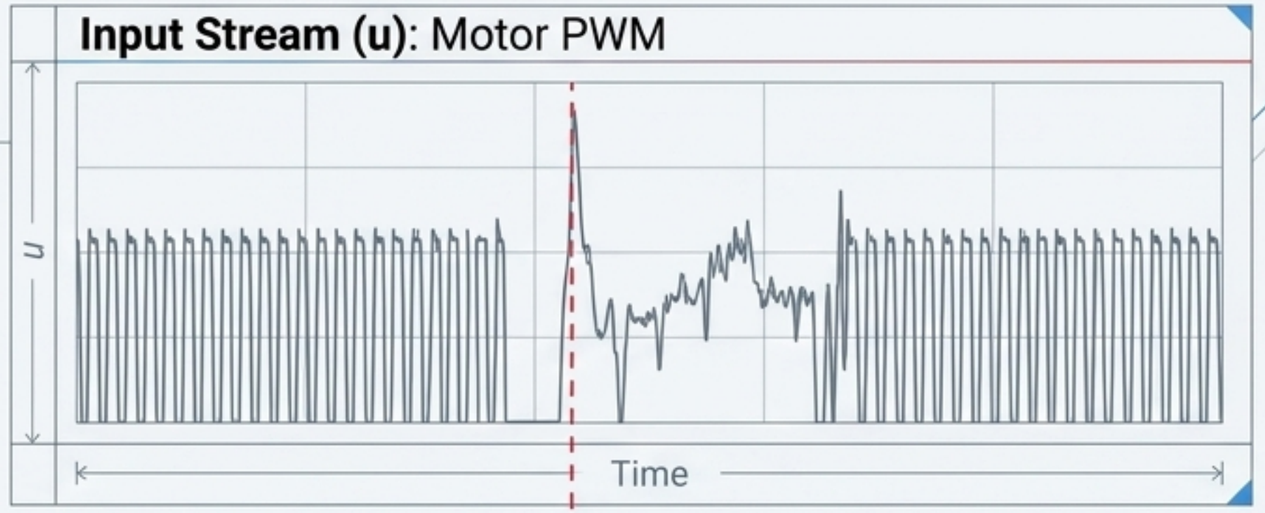
The Solution: Deploy a baseline, theoretical PID ($K_p$ =22, $K_i$ =140, $K_d$ =2) at a discrete sample time ($T_s$ = 0.01s).

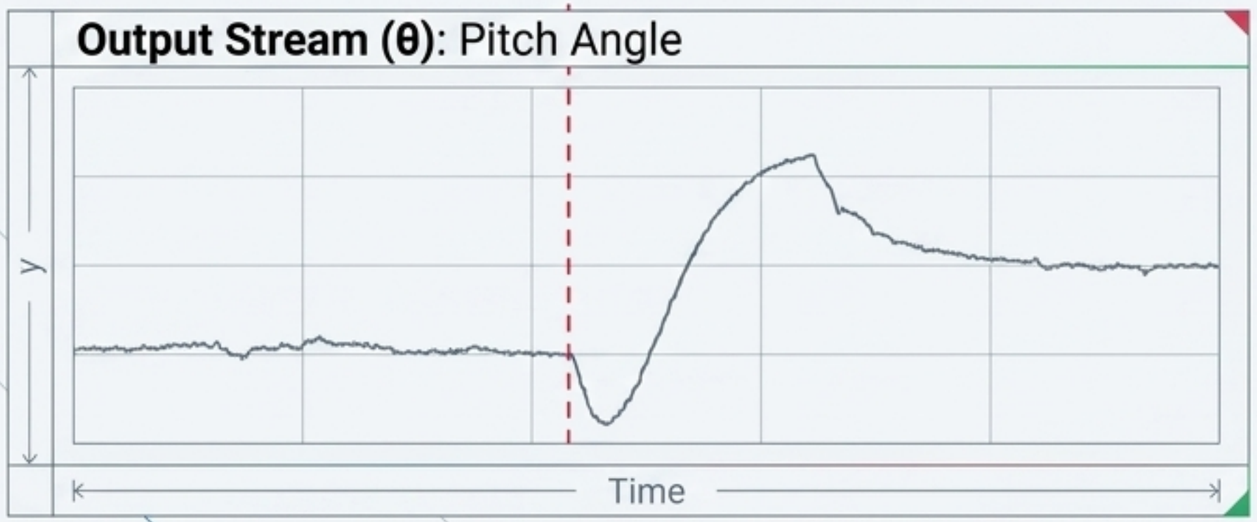
### The Purpose

Keep the hardware in the linear operating range to safely inject disturbances and log the true physical reaction.

NotebookLM

# Capturing Real-Time Hardware Telemetry




**Hardware Link:**

Simulink External Mode connected directly to the physical microcontroller.

**Input Stream (u):**

Raw motor PWM control effort sent to the L298N driver.

**Output Stream (θ):**

Physical pitch angle captured by the MPU-6050.

**Signal Processing:**

Applied deadzone removal (threshold 20), high-frequency noise smoothing (movmean 5), and detrending to yield clean System ID data.

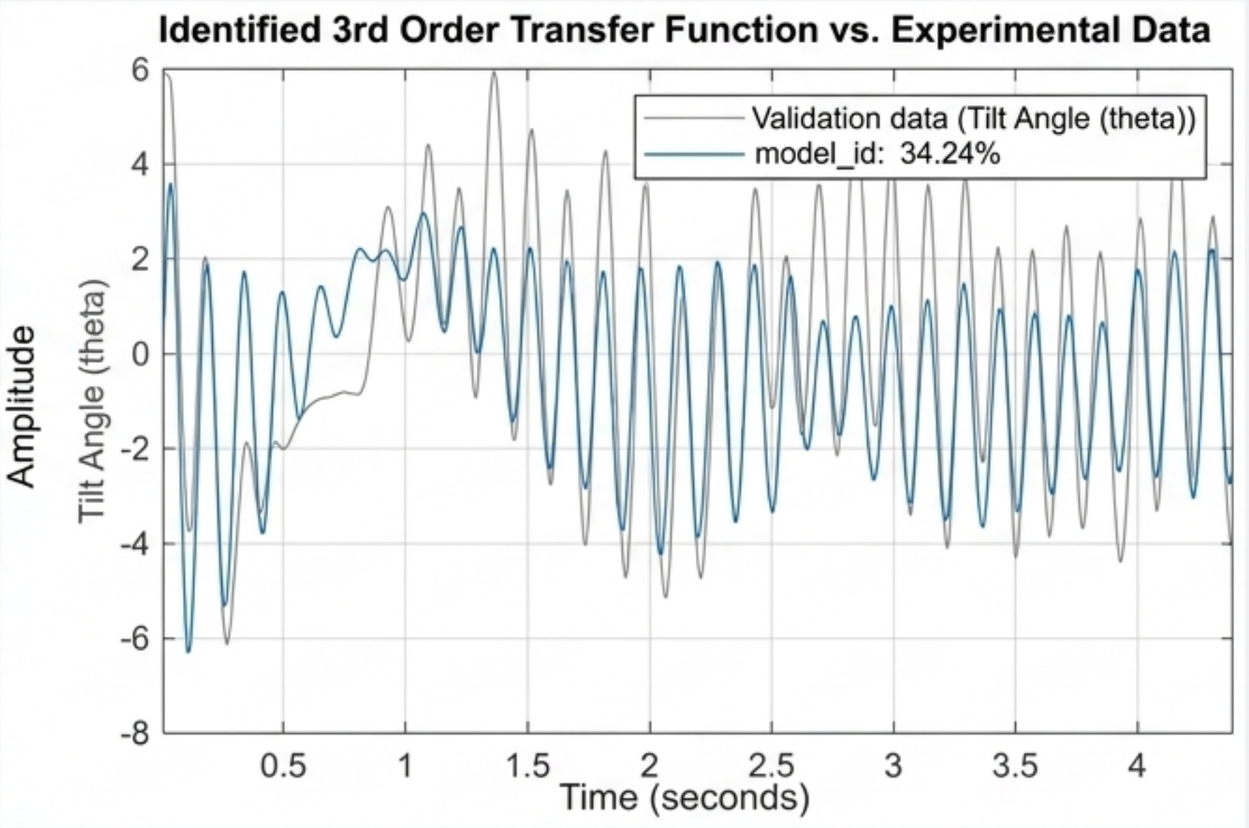
NotebookLM

# The Identified Transfer Function

Identified System:

$$\frac{148.2}{s^3 + 9.46s^2 + 1659s - 1484}$$

- **Methodology:** MATLAB TFEST tool estimating a 3-pole, 0-zero parameterization to match the theoretical model.
- **Model Fit:** 34.24% match to estimation data.




**Key Takeaway:** Successfully captured the dominant dynamic modes of a nonlinear, vibration-heavy physical environment using a 3rd-order linear approximation.

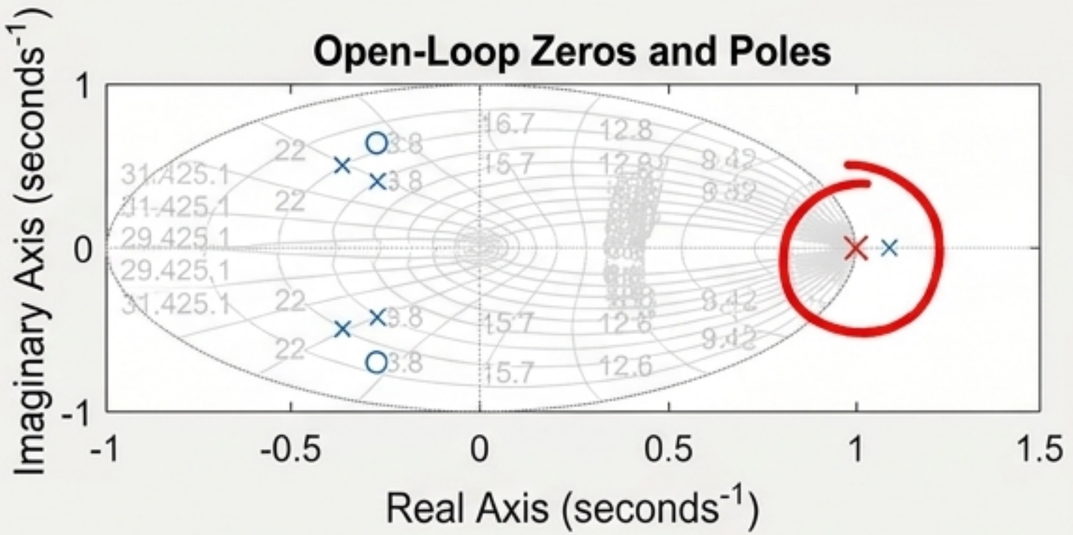
NotebookLM

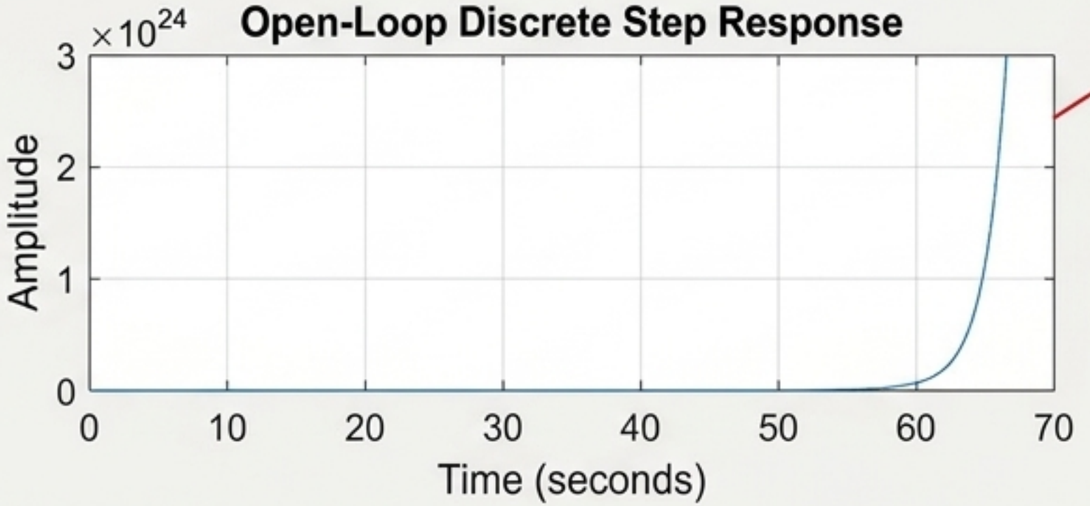
# Final Control Project Analysis

From Inherent Instability to Perfect Balance.

NotebookLM

# The Open-Loop Reality: Inevitable Failure




| System Poles: | Step Response: | Metrics: |
|---|---|---|
| Unstable right-half-plane pole at s = +8.33 rad/s. | Simulation rapidly diverges; steady-state is physically impossible. | Rise Time = **NaN**<br>Settling Time = **NaN**<br>Steady-State Error = 1.1109 |

NotebookLM

# The Final Tuned PID Controller

## Kp = 79.1355

**(The Muscle)**

Provides aggressive proportional response for immediate restoring torque against gravity.

Command window:

```
fprintf('Kp: %.4f\n', pid_tuned.Kp);
Kp: 79.1355
```



## Ki = 46.9026

**(The Stabilizer)**

Eliminates steady-state offset caused by minor physical asymmetries (e.g., center of mass misalignment).

Command window:

```
fprintf('Ki: %.4f\n', pid_tuned.Ki);
Ki: 46.9026
```



## Kd = 7.5769

**(The Brakes)**

Damps the mechanical oscillations to prevent violent over-correction and jitter.

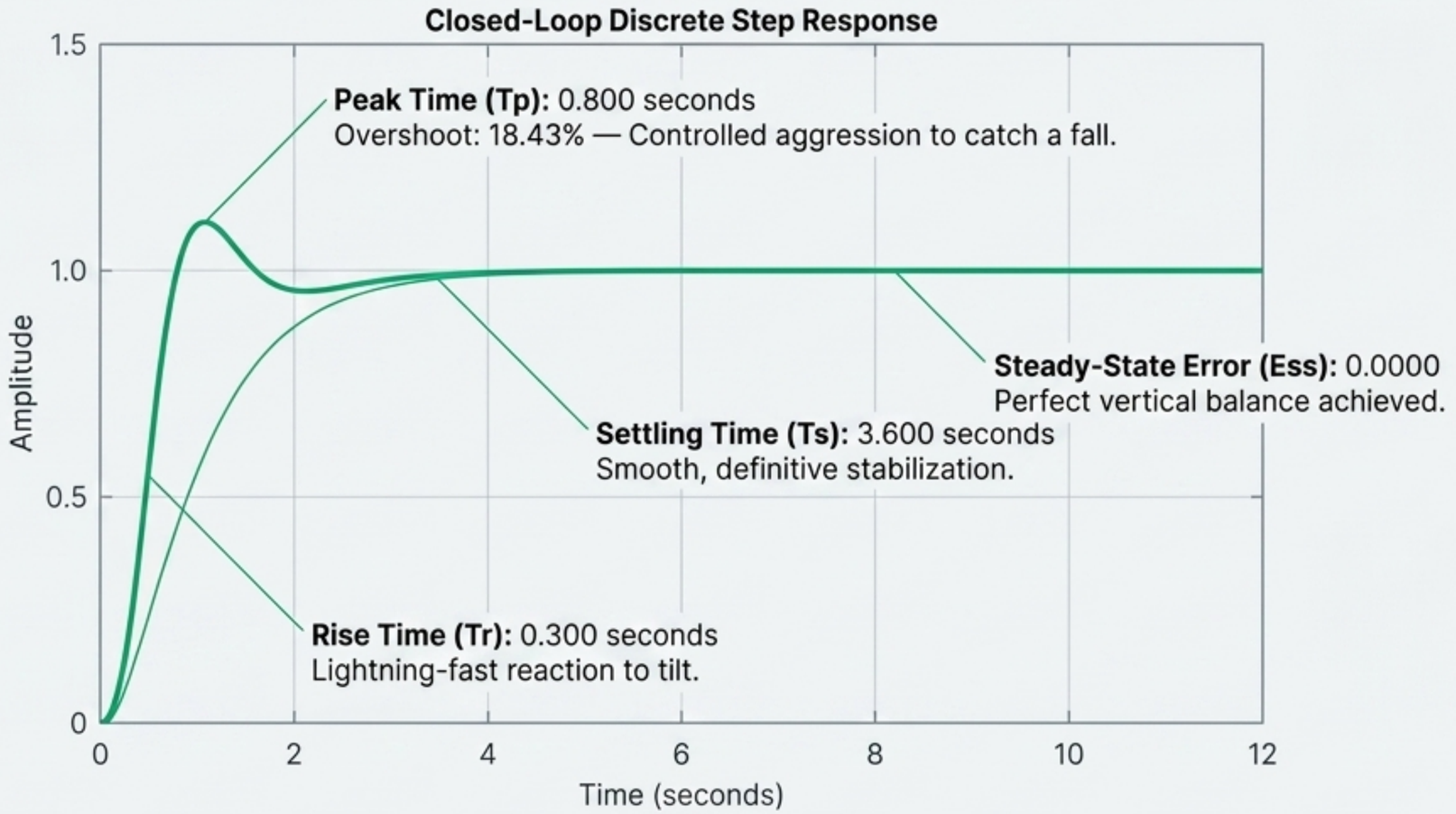
Command window:

```
fprintf('Kd: %.4f\n', pid_tuned.Kd);
Kd: 7.5769
```



NotebookLM

# Closed-Loop Performance Metrics




NotebookLM

# The Blueprint Realized

## State 1: Theoretical Plant

Modeled purely via Newton-Euler mechanics and linearized to a 3rd-order function.

## State 2: Physical Plant

Bridged via a baseline PID, yielding experimental ID data with a 34.24% TFEST fit.

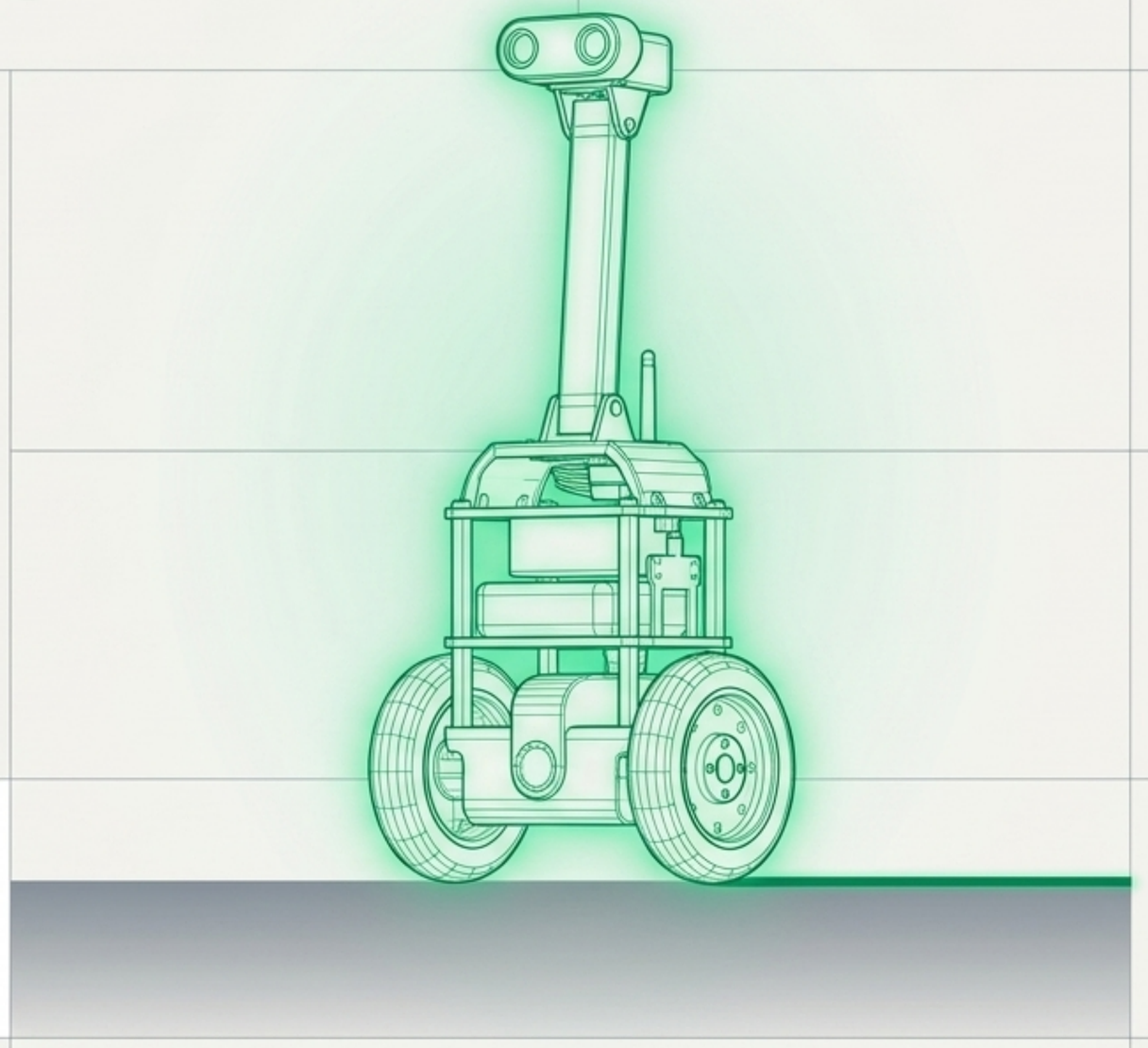
## State 3: Controlled Plant

Stabilized via a tuned PID, eliminating the rogue RHP pole and driving steady-state error to exactly zero.

NotebookLM

# Equilibrium Reached

- The inherently unstable two-wheeled inverted pendulum was successfully modeled mathematically, identified physically, and controlled digitally.
- Robust real-time balancing achieved with a settling time of 3.6s and zero steady-state error.



NotebookLM